RAPPORT DE STAGE
BTS SIO 1

Benoit DUFOUR

BTS Services Informatiques aux Organisations
Option SLAM

Entreprise d'accueil: **Gestion DS**

Du 12/05/2025 au 04/07/2025

Remerciements

Avant le développement de mon rapport de stage, j'aimerais commencer par remercier les personnes qui ont fait que ce stage a pu être possible.

Monsieur Alain DE SOUSA, président de l'entreprise pour m'avoir accepter en stage.

Monsieur Yaya WANE, mon tuteur de stage pour sa bienveillance, pour m'avoir aidé et supervisé tout au long du projet et du stage.

Sommaire

1 - Introduction 4
2 - Présentation de l'entreprise 5
2.1 - Identité juridique et informations générales 5
2.2 - Historique et secteur d'activité 5
2.3 - Organisation interne 5
3 - Environnement de travail 6
3.1 - Environnement technologique 6
3.2 - Environnement organisationnel 7
4 - Projet réalisé 8
4.1 – Objectifs et organisation 8
4.2 – Base de données 9
4.3 – Principales réalisations 10
4.4 – Difficultés rencontrées 14
5 - Savoirs et compétences développées 15
6 - Conclusion 16
7 - Annexes 17

1 - Introduction

Dans le cadre de ma première année de BTS Services informatiques aux Organisations (SIO) option Solutions Logicielles et Applications Métiers (SLAM), j'ai effectué un stage de huit semaines au sein d'une entreprise dans le but de valider mon année de formation et de mettre en pratique les compétences acquises durant cette dernière.

C'est en effet au sein du service informatique de l'entreprise Gestion DS à Villeneuve-Saint-Georges que j'ai réalisé mon stage du 12 mai 2025 au 4 juillet 2025.

Durant cette période de stage, j'ai pu mener à bien le développement complet d'un projet que le service informatique n'avait jamais eu le temps de réaliser, en raison de ses nombreuses priorités. Ce projet m'a donc permis d'acquérir et de mettre en pratique tout un tas de connaissances, tout en contribuant aux besoins qui étaient demandés.

Dans ce rapport, je vais d'abord présenter l'entreprise et son organisation, puis l'environnement technique et organisationnel de celle-ci. Viendra ensuite une présentation du projet du descriptif voulu au résultat obtenu, en passant par les difficultés rencontrées et comment je les ai surmontées. Enfin, une analyse personnelle sur les savoirs et compétences que j'ai développées tout au long de mon stage.

Pour finir, une conclusion et des annexes seront mises en place pour conclure sur cette expérience.

2 - Présentation de l'entreprise

2.1 - Identité juridique et informations générales

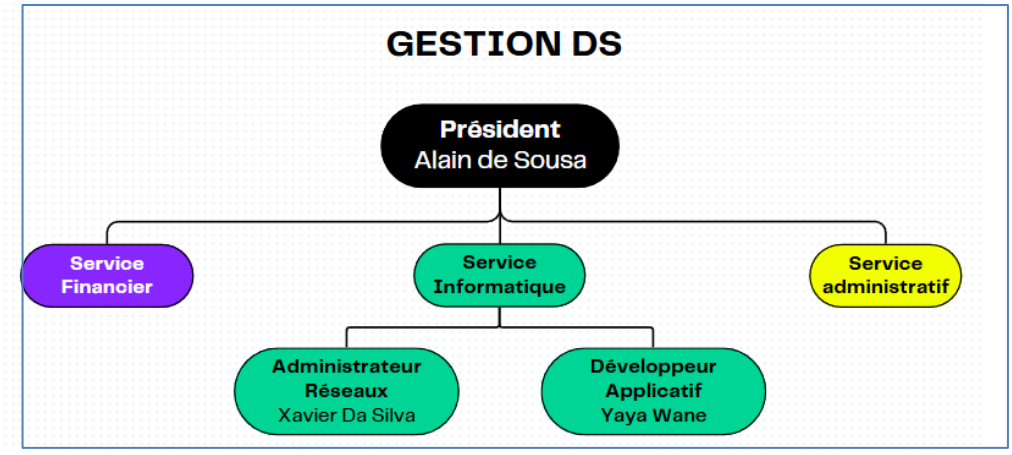
L'entreprise Gestion DS est une Société par Actions Simplifiée (SAS) immatriculée au Registre du Commerce et des Sociétés (RCS) de Créteil depuis le 1er janvier 1975. Son siège social est situé au 6 rue des Prés de l'Hôpital à Villeneuve-Saint-Georges. Gestion DS possède un capital social de 228 673.53€, un effectif de 43 salariés ainsi qu'un chiffre d'affaires (CA) de 8.8 millions d'euros en 2023.

2.2 - Historique et secteur d'activité

Créée en 1975, Gestion DS s'est développée pour devenir une structure polyvalente, axée sur la gestion administrative, financière, technique et stratégique des différentes entités des filiales. Aujourd'hui, Gestion DS occupe un rôle clé dans son secteur en agissant comme une holding interne et en proposant une gestion centralisée.

2.3 - Organisation interne

Gestion DS est une PME à taille humaine, qui est organisée en plusieurs services :



3 - Environnement de travail

3.1 – Environnement technologique

L'ensemble des logiciels, bases de données et sauvegardes utilisés de l'entreprise Gestion DS est situé, dans une armoire à serveurs sécurisée dans une pièce à température ambiante et un lecteur de pass pour pouvoir y entrer, que seul le service informatique et le directeur possèdent.

En cas d'incidents sur les différents serveurs principaux, des sauvegardes sont automatiquement effectuées.

Les différentes sauvegardes pourront donc en cas d'incidents prendre le relais et ainsi assurer la disponibilité de l'ensemble des services qui sont mis à disposition.

Lors de mon stage j'ai évolué avec la configuration ci-dessous :

- **Processeur** : Intel Core i5-7260U
- **Mémoire** : 8Go de RAM
- **HDD** : Disque dur de 250Go
- **Écran** : iiyama 21.5 pouces, 1920x1080 en 60Hz
- **Système d'exploitation** : Windows 10
- **Réseau** : Réseau fibre interne Wifi ou Ethernet propre au service informatique

3.2 – Environnement organisationnel

Le service informatique dans lequel j'ai évolué durant ces deux mois de stage est constitué d'un administrateur système et réseaux ainsi qu'un développeur applicatif. Chacun d'entre eux possède un rôle crucial et important au sein de l'entreprise.

L'administrateur système et réseaux va permettre de mettre en œuvre des solutions techniques afin d'assurer le bon fonctionnement, la disponibilité et la sécurité de l'ensemble de l'infrastructure informatique. Il est chargé de configurer et superviser les différents serveurs, le réseau ainsi que l'ensemble des postes de travails et des équipements qui vont avec.

Le développeur applicatif a lui pour but la conception, le développement et la maintenance des outils et logiciels internes afin de répondre aux besoins spécifiques de l'entreprise et d'optimiser les processus afin que l'entreprise ait le meilleur rendement possible. Il analyse les différentes demandes des autres salariés de l'entreprise qui utilisent les logiciels afin de pouvoir les créer selon leurs besoins ou encore les améliorés.

4 – Projet réalisé

4.1 – Objectifs et organisation

Le projet a pour objectif principal de développer une application qui sera capable de surveiller l'accessibilité et la disponibilité des infrastructures de l'entreprise, notamment les serveurs et les bases de données. L'ensemble de ce logiciel sera réalisé avec le Framework WINDEV qui utilise le WLangage pour programmer.

Ce logiciel permettra de :

- Vérifier périodiquement la disponibilité des serveurs et bases de données sélectionnées ;
- Détecter automatiquement les défaillances lors des contrôles périodiques ;
- Générer des rapports d'incidents et de résolution d'incidents et les envoyer par email aux personnes qui sont sélectionnées dans le logiciel;

L'organisation du projet s'est déroulée en plusieurs « modules » de tâches à effectuer et tout cela pouvait être suivis avec un diagramme de Gantt qui avait été réalisé avant mon arrivé en stage.

TÂCHE	AVANCEMENT	DÉBUT	FIN
Accueil et Découvertes			
Mini Formation WINDEV	100%	12/5/25	13/5/25
Prise de connaissance du projet	100%	14/5/25	14/5/25
Initialisation du Projet			
Création du projet	100%	15/5/25	15/5/25
Création Page principale du projet	100%	15/5/25	15/5/25
Module de logging	100%	16/5/25	16/5/25
Création de l'analyse	100%	19/5/25	19/5/25
Module de connexion à la base de donnée	100%	20/5/25	20/5/25
Gestion des Serveurs			
Création des procédures CRUD	100%	21/5/25	27/5/25
Création de la fenêtre Liste	100%	28/5/25	28/5/25
Création de la fenêtre Détails	100%	2/6/25	3/6/25

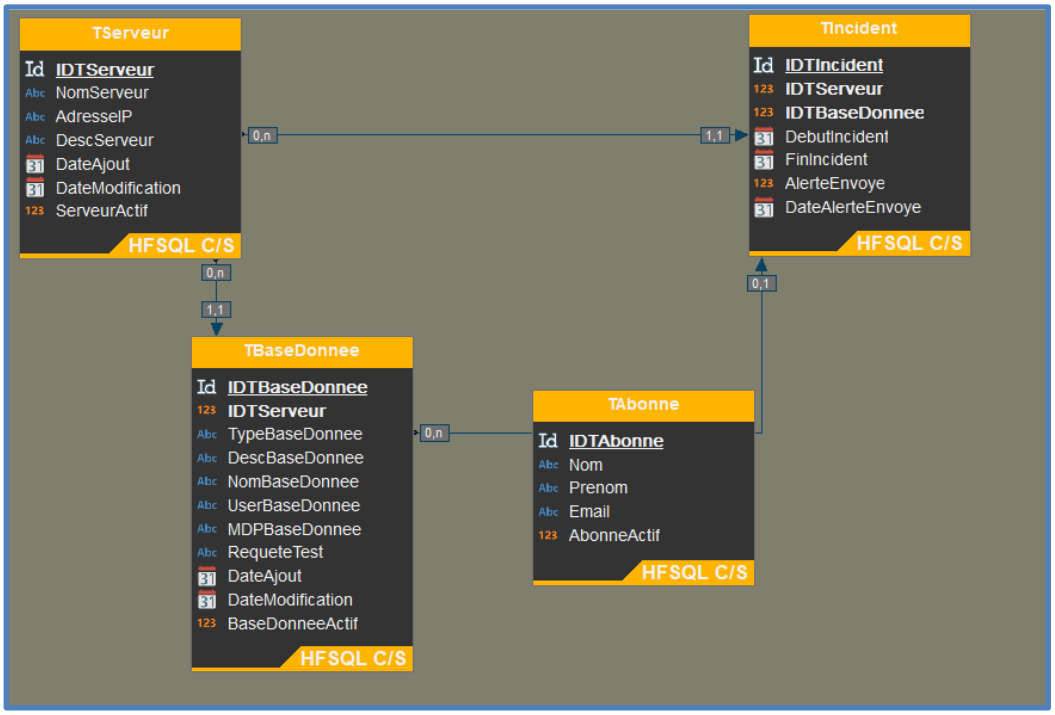
TÂCHE	AVANCEMENT	DÉBUT	FIN
Gestion des Bases de données			
Création des procédures CRUD	100%	30/5/25	4/6/25
Création de la fenêtre Liste	100%	5/6/25	5/6/25
Création de la fenêtre Détails	100%	6/6/25	9/6/25
Gestion des Abonnés			
Création des procédures CRUD	100%	10/6/25	11/6/25
Création de la fenêtre Liste	100%	12/6/25	12/6/25
Création de la fenêtre Détails	100%	13/6/25	13/6/25
Gestion des Incidents			
Incidents Serveur	100%	16/6/25	20/6/25
Incidents Base de Données	100%	23/6/25	27/6/25
Module Batch	100%	30/6/25	31/6/25
Fenêtre Tableau de Bord	100%	31/6/25	4/7/25

4.2 – Base de données

Le Framework WINDEV possède un éditeur d'analyse intégré qui lui permet de créer les bases de données avec le schéma conceptuel que nous avons créé.

Le dictionnaire de données avait déjà été réalisé à l'avance pour faciliter la réflexion lors de la création.

Cette base de données va permettre de récupérer les différentes informations qui seront stockées dans notre logiciel comme la liste des abonnés auxquels il faudra envoyer les rapports d'incidents ou encore la liste des serveurs et des bases de données dans lesquels on souhaite surveiller la disponibilité.



4.3 – Principales réalisations

A – Module de connexion à la base de données

Afin de pouvoir par la suite récupérer et ajouter les données depuis le logiciel, il a fallu que je fasse une fonction (une fonction est une procédure dans Windev) qui permettra de se connecter à la base de données. Toutes les informations de la connexion à la BDD seront situées dans un fichier de configuration .INI afin que leurs changements soient le plus simple possible et sans avoir besoin systématiquement d'entrer dans le code du projet si le nom ou encore l'IP de la base change. Cette procédure sera donc appelée à chaque lancement du logiciel.

Code de la procédure InitConnexion :

```
WL Procédure globale InitConnexion
PROCÉDURE InitConnexion()


 sServeur est une chaîne = INILit("HFSQL", "Serveur", "",sFichierIni)
 sPortTxt est une chaîne = INILit("HFSQL", "Port", "",sFichierIni)
 sBase    est une chaîne = INILit("HFSQL", "Base", "",sFichierIni)
 sUser    est une chaîne = INILit("HFSQL", "User", "",sFichierIni)
 sMdp  est une chaîne = INILit("HFSQL", "Mdp", "",sFichierIni)

SI sServeur = "" OU sPortTxt = "" OU sBase = "" OU sUser = "" ALORS
    Erreur("Le fichier ConfigServeur.ini est incomplet.")

FIN

SI PAS EstNumérique(sPortTxt) ALORS
    Erreur("Port invalide dans l'INI.")
    FinProgramme()
SINON
    nPort est un entier = Val(sPortTxt)
FIN


HDécritConnexion("MaConnexion",sUser,sMdp,sServeur + ":" + sPortTxt,sBase,hAccèsHFClientServeur,hOLectureEcriture,"")

SI HOuvreConnexion("MaConnexion") = Faux ALORS
    Info("La connexion a échoué" + ErreurInfo())
FIN
```

B – Modules de gestion des bases de données et des serveurs

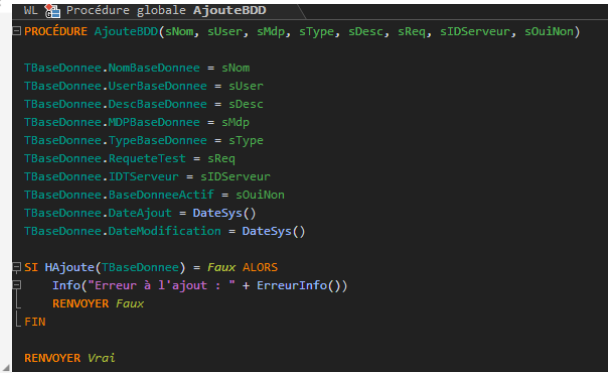
Afin de pouvoir surveiller les serveurs et bases de données que l'on souhaite, il fallait d'abord des pages qui permettraient d'indiquer toutes les informations nécessaires pour pouvoir les enregistrer dans notre logiciel comme notamment l'adresse IP sur laquelle se trouve le serveur, le mot de passe afin de pouvoir se connecter ainsi que toutes les autres informations qui seront nécessaires à la connexion.

Chacun des deux modules (serveurs et bases de données) possède une page avec des boutons CRUD qui ouvrent un formulaire afin de pouvoir y ajouter des bases ou serveurs à surveiller. Il est également possible d'en supprimer ou encore les modifier pour qu'ils ne soient plus sous surveillance.

Tableau de bord des BDD qui sont surveillées :

Formulaire d'ajout d'une BDD :

Code de la procédure AjouteBDD :

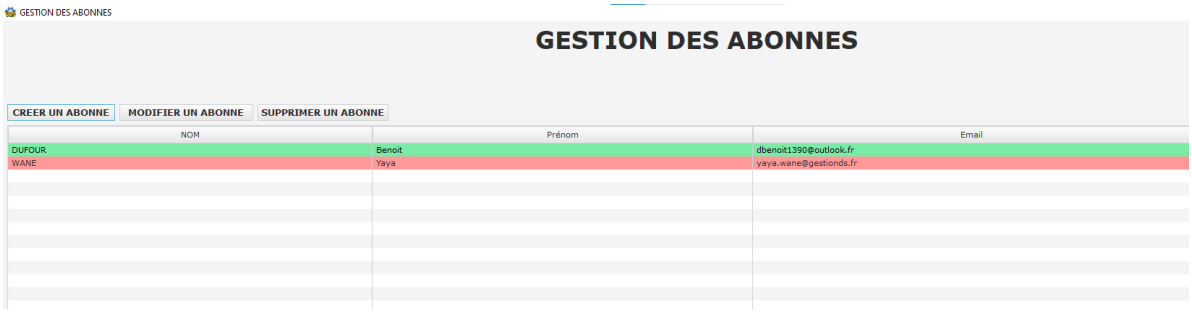


```
Procédure globale AjouteBDD
PROCÉDURE AjouteBDD(sNom, sUser, sMdp, sType, sDesc, sReq, sIDServeur, sOuiNon)

  TBaseDonnee.NomBaseDonnee = sNom
  TBaseDonnee.UserBaseDonnee = sUser
  TBaseDonnee.DescBaseDonnee = sDesc
  TBaseDonnee.MDPBaseDonnee = sMdp
  TBaseDonnee.TypeBaseDonnee = sType
  TBaseDonnee.RequeteTest = sReq
  TBaseDonnee.IDTServeur = sIDServeur
  TBaseDonnee.BaseDonneeActif = sOuiNon
  TBaseDonnee.DateAjout = DateSys()
  TBaseDonnee.DateModification = DateSys()

SI HAjoute(TBaseDonnee) = Faux ALORS
    Info("Erreur à l'ajout : " + ErreurInfo())
    RENVOYER Faux
FIN

RENVOYER Vrai
```

C – Module de gestion des abonnés

Un abonné est une personne dont l'on va venir entrer les informations dans le logiciel tel que son nom, son prénom et son adresse email qui permettra de par la suite recevoir les différents rapports. Un abonné peut être désactivé ou supprimer à tout moment depuis les boutons CRUD.

Tableau de bord des BDD qui sont surveillées :



Formulaire d'ajout d'un abonné :

D – Module de gestion d'incidents serveurs et bases de données

Lorsqu'un serveur ou une base de données est inaccessible, le module de gestion des incidents est là pour permettre la création d'un rapport d'incident qui sera répertorié sur le logiciel dans deux états possibles, en cours, non résolu (en rouge) et résolu en vert.

Les rapports d'incidents et de résolution d'incidents sont envoyés par email aux abonnés qui sont actifs avec une procédure qui va permettre l'envoi via un serveur SMTP interne à l'entreprise.

Code de la procédure EnvoieMail :

```
WL  Procédure globale EnvoieMail
PROCÉDURE EnvoieMail(sSujet est une chaîne,sRapport est une chaîne)

 MaSession est une emailSessionSMTP

 MaSession.AdresseServeur = INILit("MAILSMTP", "Serveur", "",sFichierIni)
 EmailOuvreSession(MaSession)

 MonMessage est un Email

 MonMessage.Expediteur = INILit("MAILSMTP", "Expediteur", "",sFichierIni)

 HLitPremier(TAbonne)
TANTQUE PAS HEnDehors(TAbonne)
    SI TAbonne.AbonneActif = Vrai ALORS
        Ajoute(MonMessage.Destinataire,TAbonne.Email)
    FIN
    HLitSuivant(TAbonne)
FIN

 MonMessage.Sujet = sSujet
 MonMessage.Message = sRapport

SI EmailEnvoieMessage(MaSession,MonMessage) = Faux ALORS
    RENVOYER Faux
SINON
    RENVOYER Vrai
FIN
```

4.4 – Difficultés rencontrées

L'un des premiers problèmes rencontrés concernait la connexion à la base de données du logiciel. En effet, malgré une configuration correcte des paramètres, la connexion restait impossible. Après analyse, la cause provenait d'une incompatibilité entre les versions de WinDev utilisées : celle installée sur mon poste n'était pas la même que celle utilisée par mon maître de stage. Cette différence rendait la communication avec la base de données impossible. La solution a donc été de mettre à jour mon environnement de travail afin d'utiliser la même version que celle de mon maître de stage, ce qui a immédiatement résolu le problème.

Un autre problème est survenu au niveau de la gestion des incidents. Lorsqu'un incident était détecté, le logiciel en créait un nouveau à chaque vérification, ce qui entraînait l'envoi répété de nombreux mails pour le même problème. Cette redondance nuisait à la lisibilité des alertes et surchargeait inutilement la messagerie. Pour résoudre ce dysfonctionnement, j'ai introduit une variable «null» dans le cas où il n'y en a pas en cours et permettant de vérifier si un incident concernant ce serveur était déjà enregistré. Si un incident existait déjà et était en cours (valeur «faux»), aucun nouvel incident n'était créé. Ainsi, le système enregistre un seul incident par problème réel et évite les envois multiples de mails.

Code de la procédure AjouteIncidentServ :

```
WL Procédure globale AjouteIncidentServ
PROCÉDURE AjouteIncidentServ(nIdServ,sErreur)

 bEnCours est un booléen = Faux

 HLitPremier(TIncident)
TANTQUE PAS HEnDehors(TIncident)
    SI (TIncident.IDTServeur = nIdServ) ET (TIncident.FinIncident = Null) ALORS
        bEnCours = Vrai
        SORTIR
    FIN
    HLitSuivant(TIncident)
FIN

SI bEnCours = Faux ALORS
    TIncident.IDTServeur = nIdServ
    TIncident.IDTBaseDonnee = Null
    TIncident.DebutIncident = DateHeureSys()
    TIncident.FinIncident = Null
    SI EnvoieMail("[Incident Serveur] - " + TServeur.NomServeur,"Incident sur le serveur " + TServeur.NomServeur + RC + "Détail de l'erreur : " + RC + sErreur) = Faux ALORS
        TIncident.AlerteEnvoye = Faux
        TIncident.DateAlerteEnvoye = Null
        HAjoute(TIncident)
        TableAffiche(FEN_INCIDENTSLISTE.TABLE_TIncident, taRéExécuteRequete)
    SINON
        TIncident.AlerteEnvoye = Vrai
        TIncident.DateAlerteEnvoye = DateHeureSys()
        HAjoute(TIncident)
        TableAffiche(FEN_INCIDENTSLISTE.TABLE_TIncident, taRéExécuteRequete)
    FIN
FIN
```

5 - Savoirs et compétences développées

Lors de ce stage, j'ai eu l'occasion d'obtenir de nouvelles compétences et de nouveaux savoirs.

En effet, j'ai durant ce stage développé mes compétences en développement en apprenant un nouveau langage de programmation qui est le WLangage, ce dernier et assez simple de compréhension du fait de sa compatibilité et adaptabilité avec les différentes langues et un format qui se rapproche du langage réel.

J'ai également appris à utiliser le Framework Windev 23 qui possède beaucoup d'outils à disposition comme notamment l'intégration d'outils de gestion de bases de données ou encore les liaisons avec différents services comme SMTP que j'ai donc utilisé dans le projet.

Ensuite, j'ai acquis une meilleure autonomie du fait de la découverte de ce nouveau langage avec lequel j'ai donc dû aller sur des forums ou sur la documentation technique afin de trouver certaines idées et solutions à mes problèmes.

En ce qui concerne les savoirs développés, j'ai également amélioré ma façon de réfléchir et comment faire face à un problème, plutôt que d'attendre une solution, je vais tout chercher par moi-même et demander de l'aide si cela devient nécessaire afin de ne pas rester bloquer trop longtemps sur le problème et ainsi pouvoir avancer dans les projets.

6 – Conclusion

Ce stage de huit semaines au sein de l'entreprise Gestion DS a constitué une expérience enrichissante, tant sur le plan professionnel que personnel. J'ai pu mettre en pratique les compétences acquises lors de ma formation en BTS SIO option SLAM, tout en découvrant de nouvelles technologies et méthodes de travail.

La réalisation du projet m'a permis d'approfondir mes connaissances en développement, en particulier avec l'utilisation du Framework WinDev et du WLangage et tout en développant des capacités d'autonomie et de résolution de problèmes. Les difficultés rencontrées, comme ceux liés à la connexion de la base de données, de la suppression des serveurs ou encore de la gestion des incidents, ont été une d'occasion de progresser et d'apprendre à surmonter des situations concrètes.

Ce stage m'a également permis de mieux comprendre le fonctionnement d'un service informatique en entreprise, notamment l'importance de la rigueur, de l'organisation et de la communication avec ses collaborateurs. Ces qualités seront essentielles dans la suite de mon parcours professionnel.

En définitive, ce stage a confirmé mon choix d'orientation vers le développement applicatif et renforcé ma motivation à poursuivre dans cette voie. L'expérience acquise, tant dans le développement que dans l'administration de bases de données, me servira de base solide pour mes prochaines missions et pour la suite de ma formation.

7 – Annexes

Diagramme de Gantt complet :

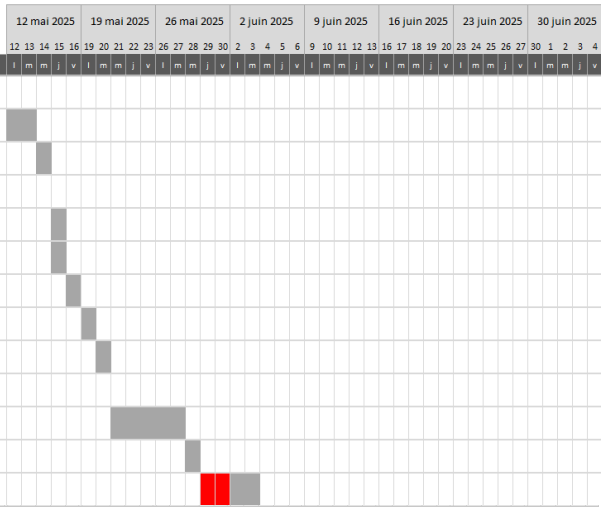
Logiciel de Surveillance d'Activités

GESTION DS

Début du projet : lun, 5/12/2025

Semaine d'affichage : 1

TÂCHE	AVANCEMENT	DÉBUT	FIN
Accueil et Découvertes			
Mini Formation WINDEV	100%	12/5/25	13/5/25
Prise de connaissance du projet	100%	14/5/25	14/5/25
Initialisation du Projet			
Création du projet	100%	15/5/25	15/5/25
Création Page principale du projet	100%	15/5/25	15/5/25
Module de logging	100%	16/5/25	16/5/25
Création de l'analyse	100%	19/5/25	19/5/25
Module de connexion à la base de donnée	100%	20/5/25	20/5/25
Gestion des Serveurs			
Création des procédures CRUD	100%	21/5/25	27/5/25
Création de la fenêtre Liste	100%	28/5/25	28/5/25
Création de la fenêtre Détails	100%	2/6/25	3/6/25



Logiciel de Surveillance d'Activités

GESTION DS

Début du projet : lun, 5/12/2025

Semaine d'affichage : 1

TÂCHE	AVANCEMENT	DÉBUT	FIN
Gestion des Bases de données			
Création des procédures CRUD	100%	30/5/25	4/6/25
Création de la fenêtre Liste	100%	5/6/25	5/6/25
Création de la fenêtre Détails	100%	6/6/25	9/6/25
Gestion des Abonnés			
Création des procédures CRUD	100%	10/6/25	11/6/25
Création de la fenêtre Liste	100%	12/6/25	12/6/25
Création de la fenêtre Détails	100%	13/6/25	13/6/25
Gestion des Incidents			
Incidents Serveur	100%	16/6/25	20/6/25
Incidents Base de Données	100%	23/6/25	27/6/25
Module Batch	100%	30/6/25	31/6/25
Fenêtre Tableau de Bord	100%	31/6/25	4/7/25

Code de l'initialisation au lancement du logiciel :

```
Initialisation de Projet Surveillance *        Si Erreur : par programme  Quand E
nServeur est un entier
nBDD est un entier
nAbonne est un entier

sFichierIni est une chaîne = ComplèteRep(fRepExe())+"/Projet_Surveillance.ini"

SI PAS fFichierExiste(sFichierIni) ALORS

    INIEcrit("GENERAL","FichierLog"," " + RC,sFichierIni)


    INIEcrit("HFSQL","Serveur "," localhost",sFichierIni)
    INIEcrit("HFSQL","Port "," 4900",sFichierIni)
    INIEcrit("HFSQL","Base "," Surveillance",sFichierIni)
    INIEcrit("HFSQL","User "," admin",sFichierIni)
    INIEcrit("HFSQL","Mdp "," " + RC,sFichierIni)

    INIEcrit("MAILSMTP","Serveur","10.94.1.25",sFichierIni)
    INIEcrit("MAILSMTP","Port","25",sFichierIni)
    INIEcrit("MAILSMTP","Expediteur","support@gestionds.fr" + RC,sFichierIni)

FIN
InitConnexion()

TabLibelle est un tableau de chaîne = ["HFSQL", "MySQL"]

gsLigneCmd est une chaîne = LigneCommande()

SI Position("/batch",gsLigneCmd) > 0 ALORS
    TANTQUE PingServ() <> Vrai
        Temporisation(50)
    FIN
    TANTQUE VerifBases() <> Vrai
        Temporisation(50)
    FIN
        FinProgramme()
FIN
```

Procédure PingServ qui envois un ping au serveur pour vérifier s'il fonctionne :

```
WL  Procédure globale PingServ
PROCÉDURE PingServ()

 HLitPremier(TServeur)
TANTQUE PAS HEnDehors(TServeur)
    SI TServeur.ServeurActif = Vrai ALORS

        SI Ping(TServeur.AdresseIP,1000) = Faux ALORS
            sErreur est une chaîne = ErreurInfo()
            AjouteIncidentServ(TServeur.IDTServeur,sErreur)
        SINON SI Ping(TServeur.AdresseIP,1000) = Vrai ALORS
            TermineIncidentServ(TServeur.IDTServeur)
        FIN
    FIN
    HLitSuivant(TServeur)
FIN
RENVOYER Vrai
```

Procédure VérifBases qui envoie une requête SQL au serveur pour vérifier s'il fonctionne :

```
WL  Procédure globale VerifBases
PROCÉDURE VerifBases()
 bOK est un booléen
 sdResSQL est une Source de Données
 HLitPremier(TBaseDonnee)
TANTQUE PAS HEnDehors(TBaseDonnee)
    SI TBaseDonnee.BaseDonneeActif = Vrai ALORS
        SI PAS HLitRecherche(TServeur,TServeur.IDTServeur,TBaseDonnee.IDTServeur) ALORS
        SINON
            SI TBaseDonnee.TypeBaseDonnee = "MySQL" ALORS
                HDécritConnexion("MaConnexion",TBaseDonnee.UserBaseDonnee,TBaseDonnee.MDPBaseDonnee,TServeur.AdresseIP,TBaseDonnee.NomBaseDonnee,hAccèsNatifMySQL,hOLecture)
            SINON
                HDécritConnexion("MaConnexion",TBaseDonnee.UserBaseDonnee,TBaseDonnee.MDPBaseDonnee,TServeur.AdresseIP + ":4900",TBaseDonnee.NomBaseDonnee,hAccèsHFClientServeur,hOLecture)
            FIN
            SI HOuvreConnexion("MaConnexion") = Faux ALORS
                sErreur est une chaîne = ErreurInfo()
                AjouteIncidentBDD(TBaseDonnee.IDTServeur,TBaseDonnee.IDTBaseDonnee,sErreur)
            SINON
                SI HOuvreConnexion("MaConnexion") = Vrai ALORS
                    bOK = HExécuteRequêteSQL(sdResSQL,"MaConnexion",hRequêteSansCorrection,TBaseDonnee.RequeteTest)
                    sErreur est une chaîne = ErreurInfo()
                    SI bOK = Faux ALORS
                        AjouteIncidentBDD(TBaseDonnee.IDTServeur,TBaseDonnee.IDTBaseDonnee,sErreur)
                    SINON SI bOK = Vrai ALORS
                        TermineIncidentBDD(TBaseDonnee.IDTServeur,TBaseDonnee.IDTBaseDonnee)
                    FIN
                FIN
            FIN
        FIN
    FIN
    HLitSuivant(TBaseDonnee)
FIN
RENVOYER Vrai
```